ATIVIDADE AVALIATIVA – PROJETO IoT

Wokwi + PlatformIO + MQTT + Node-RED + Dashboard + HTTP + MySQL

Roteiro de desenvolvimento, documentação, testes e apresentação final

Nome do projeto	RANIKA - Central de Monitoramento Hospitalar UTI
Integrantes	Karina Moraes Garcia, Nicolly Lemos, Rafaela Mansano
Turma	Programação Multiplataforma
Professor	Prof. Emerson de Oliveira Rocha
Data da apresentação	06/11/2026

⚠ **ATUALIZAÇÃO E ENTREGA DO ROTEIRO FINAL:** Este roteiro foi completamente preenchido e atualizado pela equipe RANIKA, refletindo a implementação técnica final do projeto, incluindo simulação no Wokwi, firmware C++ no PlatformIO, barramento MQTT via HiveMQ, servidor de regras no Node-RED, API REST HTTP GET (/api/status), banco de dados MySQL no XAMPP e Dashboard interativo com aplicação de medicação gradual.

1. Objetivo da atividade

Desenvolver e documentar uma solução IoT completa de monitoramento hospitalar em tempo real para Unidades de Terapia Intensiva (UTI), integrando captação de sinais vitais no ESP32 (Wokwi/PlatformIO), protocolo de comunicação MQTT (HiveMQ), servidor de automação e triagem no Node-RED, API REST via HTTP (`GET /api/status`), armazenamento relacional no banco de dados MySQL (XAMPP) e interface gráfica interativa no Dashboard do Node-RED com controle de medicação gradual.

2. Identificação e planejamento do projeto

Descrição do problema:

Em ambientes de Unidade de Terapia Intensiva (UTI), a verificação manual intermitente dos sinais vitais dos pacientes pode gerar atrasos na identificação de picos febris repentinos ou paradas/taquicardias, colocando vidas em risco. Além disso, a ausência de um registro digital automático das ocorrências dificulta a auditoria médica e a análise do histórico clínico do paciente.

Objetivo do projeto:

Criar a Central de Monitoramento RANIKA para 4 Leitos de UTI, permitindo o acompanhamento contínuo de Temperatura Corporal (°C) e Frequência Cardíaca (BPM), a emissão de alertas sonoros/visuais instantâneos em caso de emergência, a aplicação de intervenção terapêutica (medicação gradual) via Dashboard e a gravação permanente do histórico de eventos no banco de dados MySQL.

Público-alvo e contexto de uso:

Equipes de enfermagem, médicos intensivistas e gestores hospitalares em Unidades de Terapia Intensiva, salas de emergência e centros de triagem clínica.

Especificação de Hardware, Sensores e Atuadores:

Componente / Pino	Tipo	Função no Projeto
Potenciômetro 1 (GPIO 34)	Entrada Analógica (ADC)	Simulação contínua do Sensor de Temperatura Corporal (35.0 °C a 42.0 °C).
Potenciômetro 2 (GPIO 35)	Entrada Analógica (ADC)	Simulação do Sensor de Frequência Cardíaca / ECG (40 a 160 BPM).
Botão de Emergência (GPIO 12)	Entrada Digital (Pull-down)	Acionamento manual do Alarme de Emergência Crítica pelo leito.
Buzzer Piezotérmico (GPIO 14)	Saída Digital (PWM/Tone)	Emissão do alarme sonoro de emergência clínica pulsado.
LED Vermelho (GPIO 13)	Saída Digital	Sinalização visual imediata de alerta/emergência no leito.

ETAPA 3 – SIMULAÇÃO NO WOKWI

3.1 Montagem do circuito

O circuito virtual foi montado no simulador Wokwi utilizando um microcontrolador ESP32 DevKit V1. Os dois potenciômetros analógicos foram conectados aos pinos ADC GPIO 34 e GPIO 35. O botão tátil de emergência foi ligado ao pino GPIO 12 com um resistor de pull-down de 10kΩ. O LED vermelho de alarme foi conectado ao pino GPIO 13 (com resistor limitador de corrente de 220Ω) e o Buzzer piezotérmico ao pino GPIO 14.

3.2 Teste dos sensores

Sensor / Sinal	Faixa de Valor	Unidade	Resultado / Observação Clínico-Técnica
Temperatura Corporal	35.0 a 42.0	°C	Conversão analógica perfeita. Leitura febril (>37.5°C) aciona aviso no Dashboard; febre alta (>38.5°C) aciona emergência.
Frequência Cardíaca	40 a 160	BPM	Variação contínua. Valores >100 BPM indicam taquicardia (alerta) e >120 BPM ativam a emergência crítica.
Botão de Emergência	0 ou 1	Digital	Acionamento instantâneo. Ao pressionar, dispara o alarme físico no Wokwi e gera flag imediato no JSON.

Evidências do Wokwi:

• Circuito montado e validado sem erros de firmware ou componentes soltos.

• Leitura analógica em tempo real refletida perfeitamente no terminal e na telemetria.

ETAPA 4 – DESENVOLVIMENTO NO PLATFORMIO

4.1 Desenvolvimento do código

O projeto foi estruturado no ambiente PlatformIO (VS Code) utilizando C++ com as bibliotecas **WiFi.h**, **PubSubClient.h** (comunicação MQTT) e **ArduinoJson.h** (serialização de payloads). O código realiza a leitura dos sensores a cada 2000 ms, monta o objeto JSON e o publica via MQTT no broker público HiveMQ (`broker.hivemq.com`). Além disso, escuta os tópicos de comando para acionar o LED e o Buzzer no leito.

4.2 Testes do ESP32

Teste de Validação	Resultado / Observação no PlatformIO
ESP32 iniciou corretamente	OK – Boot concluído com sucesso, com registradores configurados.
Sensores funcionando	OK – Mapeamento analógico convertido com precisão para °C e BPM.
Wi-Fi conectado	OK – Conexão automática estabelecida com o AP virtual Wokwi-GUEST.
MQTT conectado	OK – Conectado com sucesso ao broker broker.hivemq.com na porta 1883.
Monitor Serial apresentando dados	OK – Transmissão JSON contínua observada no Terminal do PlatformIO.

ETAPA 5 – CONFIGURAÇÃO DO MQTT

5.1 Tópicos MQTT do Projeto

Função do Tópico	Tópico MQTT	Direção da Mensagem
Telemetria Leito 01 (ESP32 / Wokwi)	ranika/leito01/telemetria	ESP32 ➔ Node-RED
Comando Atuador Buzzer	ranika/leito01/atuadores/buzzer	Node-RED ➔ ESP32
Comando Atuador LED	ranika/leito01/atuadores/led	Node-RED ➔ ESP32
Escuta Global dos 4 Leitos	ranika/+/telemetria	Broker ➔ Node-RED
Intervenção de Medicação Gradual	ranika/leito01/medicar	Dashboard ➔ Node-RED

5.2 Exemplo de Payload JSON Transmitido

Tópico: ranika/leito01/telemetria

Payload JSON:

```
{
  "paciente": "Ranika (Leito 01)",
```

```
  "temperatura": 39.5,
  "bpm": 132,
  "emergencia": true
}
```

ETAPA 6 – DESENVOLVIMENTO DO FLUXO NO NODE-RED, HTTP REST E MYSQL

6.1 Arquitetura do Servidor Node-RED

O Node-RED atua como a central inteligente do projeto, integrando três pilares vitais:

1. Processamento e Triagem de 4 Leitos: O nó 'Central Processadora 4 Leitos' avalia os sinais vitais recepcionados via MQTT e gera os alertas visuais (Verde = Estável, Amarelo = Febre/Taquicardia, Vermelho = Emergência Crítica).

2. Integração com Banco de Dados MySQL (XAMPP): O nó 'Montar SQL' intercepta os dados e envia requisições SQL de inserção direta para o banco de dados uti_hospitalar (porta 3306), registrando o paciente_id fixo (Ranika = 1), nome, leito, tipo de evento, temperatura, BPM e a data/hora exata do registro.

3. Endpoint de API REST HTTP GET (/api/status): O nó 'http in' disponibiliza uma interface HTTP REST na URL http://localhost:1880/api/status. Qualquer cliente ou sistema externo pode consultar via HTTP GET o estado consolidado em JSON de todos os leitos ao vivo.

6.2 Estrutura do Banco de Dados MySQL (XAMPP)

A tabela **historico_alertas** foi configurada no phpMyAdmin com a seguinte estrutura relacional:

- id: Auto-incremento (Chave Primária do log)
- paciente_id: Chave Ficha do Paciente (Ranika = ID 1, Ana Paula = ID 2, Carlos = ID 3, Mariana = ID 4)
- leito: Identificador do leito hospitalar
- paciente_nome: Nome completo do paciente
- tipo_evento: Status clínico (Ex: 'SINAIS VITAIS REGULARES', 'EMERGÊNCIA CRÍTICA', 'MEDICAÇÃO APLICADA')
- temperatura / bpm: Valores numéricos registrados
- data_hora: Timestamp automático do servidor MySQL (DATETIME)

ETAPA 7 – DESENVOLVIMENTO DO DASHBOARD E MEDICAÇÃO GRADUAL

7.1 Recursos do Dashboard (/ui)

O Dashboard do Node-RED foi customizado com estilos CSS hospitalares profissionais, apresentando:

- Central de Monitoramento Unificada cobrindo os 4 Leitos de UTI simultaneamente.
- Banners com Gradiente Dinâmico e Ícones de Saúde (Verde, Amarelo e Vermelho Pulsante).
- Gauges de Temperatura (35°C - 42°C) e Frequência Cardíaca (40 - 160 BPM).

• **Módulo Interativo de Medicação Gradual:** Ao clicar no botão '💉 APLICAR MEDICAÇÃO', o sistema dispara uma curva de resposta terapêutica realista de ~10 segundos (dividida em 3 etapas), reduzindo suavemente a febre e os batimentos até atingir 36.5 °C e 75 BPM, normalizando o leito e registrando o procedimento no MySQL.

ETAPA 8 – TESTES FINAIS E DEMONSTRAÇÃO COMPLETA

O sistema completo foi testado e validado em um cenário clínico end-to-end:

1. Teste de Operação Normal: Sinais vitais normais no Wokwi mantêm os banners verdes ('✅ SINAIS VITAIS ESTÁVEIS').

2. Simulação de Emergência Física: Pressionar o botão físico de emergência ativa o LED vermelho e o Buzzer no Wokwi, alterando o banner no Dashboard para '🚨 EMERGÊNCIA CRÍTICA' e salvando a ocorrência no MySQL.

3. Aplicação de Medicação Terapêutica: Clique no botão de medicação resulta na queda gradual da febre em 10 segundos e gravação do evento no banco de dados.

4. Validação da API REST HTTP: Acesso a http://localhost:1880/api/status retorna o JSON com o estado clínico dos 4 leitos instantaneamente.

9. RELATÓRIO FINAL DO PROJETO

9.1 Resultados Obtidos

O projeto RANIKA atingiu 100% dos seus objetivos técnicos e acadêmicos. A integração entre o ESP32 (Wokwi), o barramento MQTT (HiveMQ), o Node-RED, o MySQL (XAMPP) e a API REST HTTP funcionou de maneira impecável e robusta, garantindo um sistema completo de telemetria e gestão hospitalar.

9.2 Dificuldades Encontradas e Soluções Adotadas

1. Duplicidade Visual dos Banners no Dashboard:

Problema: O nó de CSS global estava repassando mensagens HTML para a tela do Leito 01, gerando dois banners verdes empilhados.

Solução: Ajustou-se a propriedade do nó de template CSS para 'Pass-through = false', isolando os estilos e mantendo apenas 1 banner limpo por leito.

2. Identificação Fixa dos Pacientes no Banco de Dados MySQL:

Problema: A tabela do MySQL gerava IDs incrementais para cada log de alerta, causando dúvida sobre como associar as linhas ao mesmo paciente.

Solução: Adicionou-se a coluna 'paciente_id' fixa na tabela historico_alertas. Assim, todas as ocorrências da paciente Ranika mantêm 'paciente_id = 1', mantendo o histórico de saúde organizado e auditável.

3. Queda Brusca na Simulação de Medicação:

Problema: A medicação alterava a temperatura de 39.5°C para 36.5°C de forma instantânea, o que não refletia a realidade clínica.

Solução: Implementou-se uma função assíncrona com temporizadores (setTimeout) no Node-RED, criando uma curva de transição suave em 3 etapas distribuídas em ~10 segundos.

10. TABELA DE CRITÉRIOS DE AVALIAÇÃO DO PROFESSOR

Critério de Avaliação	O que será observado	Valor	Nota
1. Planejamento e definição	Problema, objetivo, requisitos e organização inicial.	1,0	0
2. Wokwi e montagem do circuito	Circuito, sensores, atuadores e funcionamento da simulação.	1,0	0
3. PlatformIO e código ESP32	Organização do código, leitura dos sensores e funcionamento do ESP32	1,0	0
4. Comunicação MQTT	Broker, tópicos, publicação e recebimento das mensagens	1,0	0
5. Node-RED, HTTP e MySQL	Fluxo de processamento, regras e integração com os dispositivos.	1,0	0
6. Dashboard e Interatividade	Visualização, gráficos, indicadores, alertas e/ou controles.	1,0	0
7. Testes e funcionamento	Testes normais, situações de alerta e evidências dos resultados.	1,0	0
8. Documentação e roteiro	Registro contínuo, evidências, dificuldades, soluções e versão final atualizada.	1,0	0
9. Integração e criatividade	Integração das tecnologias e adequação da solução ao problema.	1,0	0
10. Apresentação	Clareza, organização, domínio do projeto e demonstração prática.	1,0	0
TOTAL	**Pontuação final da Atividade Avaliativa de Projeto IoT**	**10,0**	**0**